Todo App — Technical Document

> Attached to: docs/todo-app/user-story.md Type: architecture

Architectural Pattern

Layered RESTful microservice following a Controller → Service → Repository pattern. Single deployable Spring Boot application with no external service dependencies in this phase.

Technology Stack

Concern	Technology
Language	Java 17+
Framework	Spring Boot 3.x
Build Tool	Maven
Database (prod)	PostgreSQL
Database (test/local)	H2 (in-memory)
Schema Migration	Flyway
API Documentation	SpringDoc OpenAPI (Swagger UI)
Testing	JUnit 5, Mockito, Spring Boot Test

Component Design

Controller Layer

- `TaskController` — handles all HTTP requests for `/tasks` and `/tasks/{id}`
- Responsible for: request validation, HTTP status codes, response mapping
- No business logic — delegates entirely to service layer

Service Layer

- `TaskService` — owns all business logic
- Responsible for: CRUD orchestration, filtering, sorting, status transitions, timestamp management

- Throws typed exceptions (`TaskNotFoundException` , `ValidationException`) — never returns nulls

Repository Layer

- `TaskRepository` — extends `JpaRepository<Task, UUID>`
- Custom query methods for filtering by `status` , `priority` , `label`
- Sorting handled via `Sort` parameter passed from service layer

Domain Model

- `Task` — JPA entity mapped to `tasks` table
- `TaskStatus` — enum: `ACTIVE` , `COMPLETED`
- `TaskPriority` — enum: `HIGH` , `MEDIUM` , `LOW`

DTOs

- `CreateTaskRequest` — input for POST, validates `title` required, max lengths
- `UpdateTaskRequest` — input for PUT, all fields optional
- `PatchStatusRequest` — input for PATCH `/status`
- `TaskResponse` — output for all endpoints, never exposes entity directly

Exception Handling

- `GlobalExceptionHandler` (`@RestControllerAdvice`) — catches all exceptions
- Returns consistent error structure: `{ "error": "ERROR_CODE", "message": "..." }`
- `404` for `TaskNotFoundException` , `400` for validation failures

API Design

Base path: `/tasks` Versioning: none in this phase

Method	Path	Description
POST	`/tasks`	Create task
GET	`/tasks`	List with optional filters and sort
GET	`/tasks/{id}`	Get by ID
PUT	`/tasks/{id}`	Full update
DELETE	`/tasks/{id}`	Delete permanently
PATCH	`/tasks/{id}/status`	Toggle status

Query Parameters for GET /tasks

- `status` — `ACTIVE` or `COMPLETED`
- `priority` — `HIGH`, `MEDIUM`, or `LOW`
- `label` — free-text string match
- `sortBy` — `createdAt` (default), `priority`, `title`
- `sortDir` — `asc` or `desc` (default `desc`)

Security

No authentication required in this phase. All endpoints are publicly accessible. To be added in a future phase.

Observability

- Spring Boot Actuator enabled for health checks at `/actuator/health`
- Structured logging via SLF4J — log all requests at INFO, errors at ERROR
- No distributed tracing in this phase

API Documentation

- Swagger UI: `/swagger-ui.html`
- OpenAPI spec: `/v3/api-docs`
- All endpoints, request/response models, and error codes documented

Diagrams

DIA-001 — Layered Architecture

Type: c4-component Description: Shows the three-layer structure (Controller → Service → Repository) and their interactions with the PostgreSQL database.

DIA-002 — Task Lifecycle Sequence

Type: sequence Description: End-to-end sequence for creating a task — HTTP request → Controller → Service → Repository → DB → response.

DIA-003 — Entity Relationship

Type: er Description: Single `tasks` table with all fields, types, and constraints as defined in the data model.